Data dependencies

Pure/Flow dependence

```
A: ADD R1, R2, R3
B: ADD R4, R1, R3
```

RAW

False Dep

Anti-Dependence

```
A: ADD R4, R1, R3
B: ADD R1, R2, R3
```

WAR

Renaming

Output Dependence

```
A  ADD R1, R2, R3
B  ADD R1, R4, R5
```

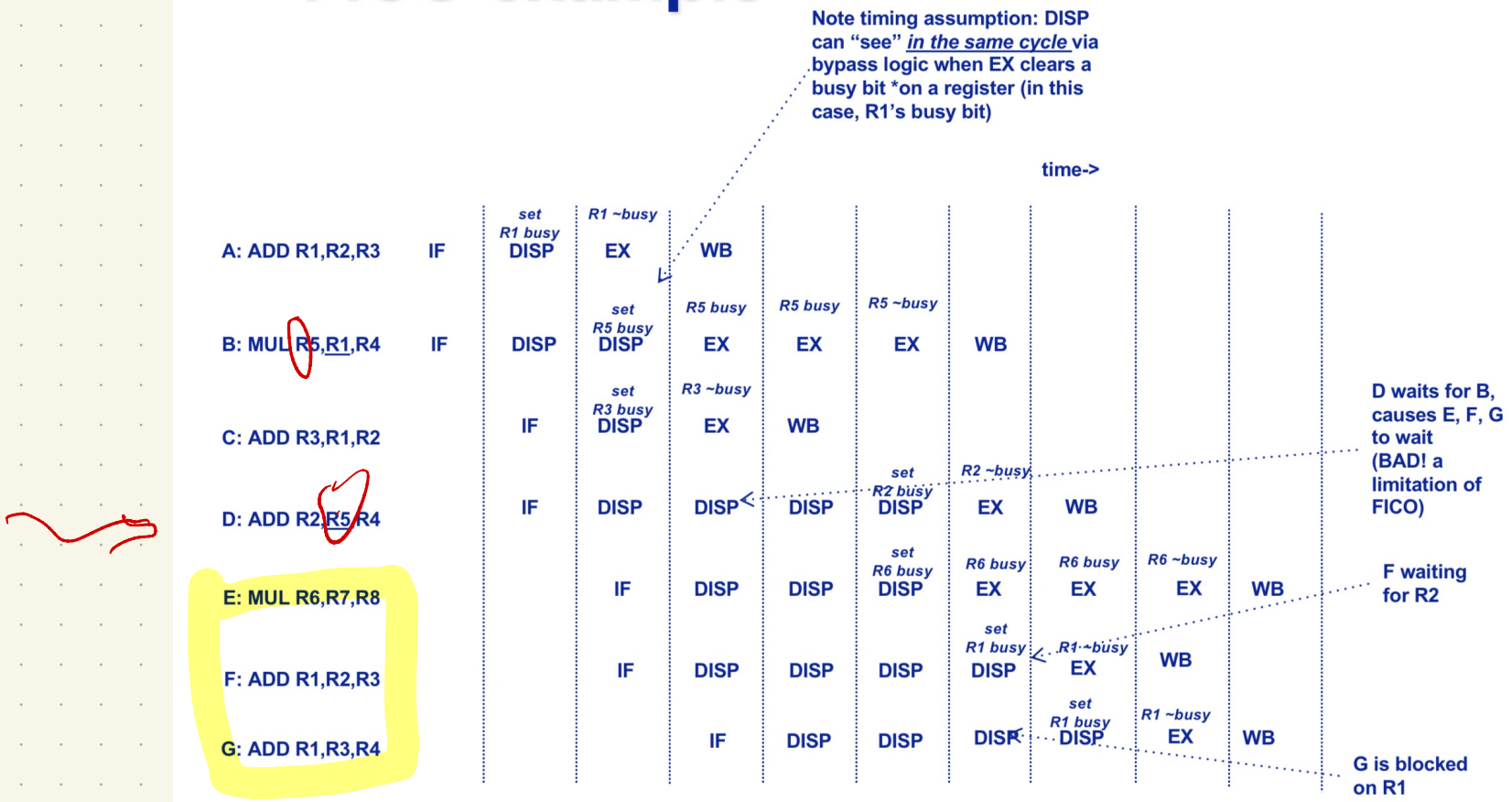
WAW

Renaming:

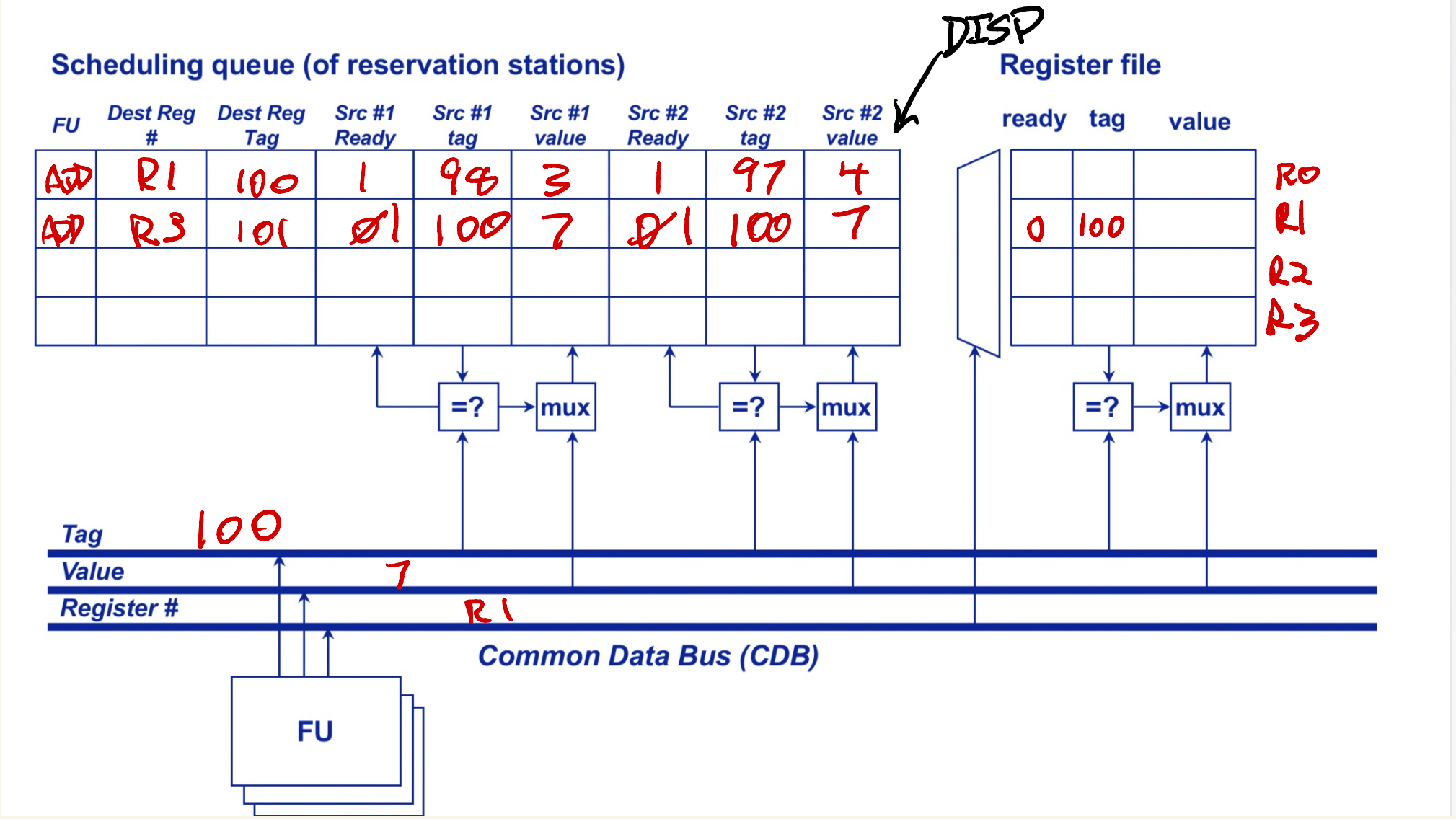
Fire out of order requirement

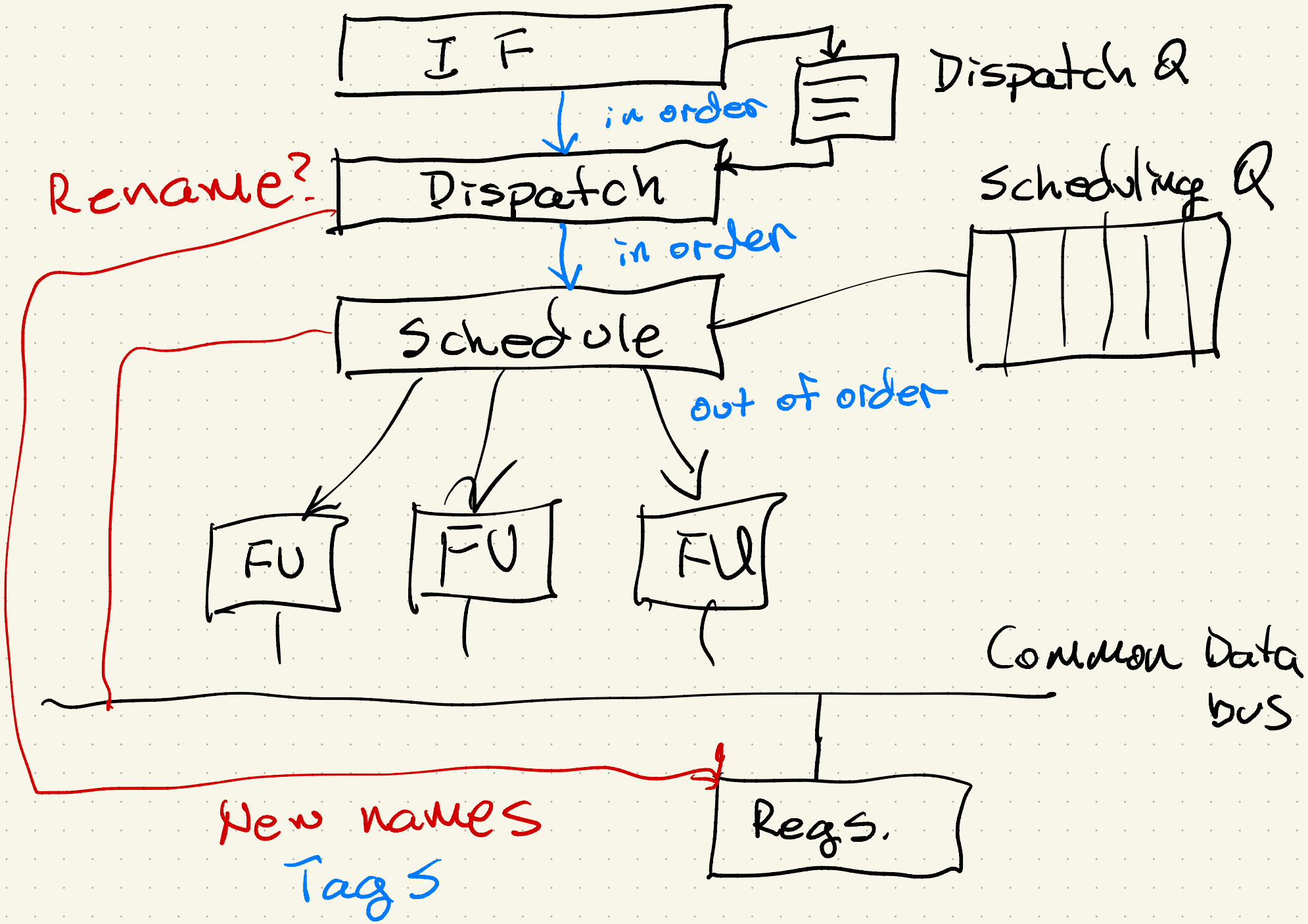
- internal competition in IBM late '60s
  - 360 model 85 - "Buffer" memory ((cache memory))
    - vs
  - 360 model 91 - Fire out of order
    - Bob Tomasulo
    - Hardware renaming for FP unit

FICO example



Safe but limiting


DISP
Scheduling queue (of reservation stations)
Register file
FU
Dest Reg #
Dest Reg Tag
Src #1 Ready
Src #1 tag
Src #1 value
Src #2 Ready
Src #2 tag
Src #2 value
ADD R1 100 1 98 3 1 97 4
ADD R3 101 0 1 100 7 0 1 100 7
ready tag value
0 100
R0
R1
R2
R3
=?
mux
=?
mux
=?
mux
Tag 100
Value 7
Register # R1
Common Data Bus (CDB)
FU


I F
Dispatch Q
in order
Rename?
Dispatch
Scheduling Q
in order
Schedule
out of order
FU
FU
FU
Common Data bus
New names
Tags
Regs.

Tomasulo algorithm, part 1

For each cycle of execution…

```
[Dispatch unit ] for all instructions I in DispQ do:
    if (SchedQ is not full) then
        (a) Add I to first free slot of SchedQ  (="RS")      // Call this RS (for "reservation station") below
        (b) Delete I from DispQ                               // Insttruction I is now "dispatched"
        (c) RS.FU = I.FU
        (d) RS.Dest = I.Dest
        (e) for i = 0..1 do
                if (Regs[I.src[i]].Ready = True) then
                        RS.Src[i].Value = Regs[I.src[i]].Value
                        RS.Src[i].Ready = True
                else
                        RS.Src[i].Tag = Regs[I.src[i]].Tag  // Copy the tags from the RF
                        RS.Src[i].Ready = False
        (f) Regs[I.Dest].Tag = unique tag id    // tag (rename) the destination
        (g) RS.DestRegTag = Regs[I.Dest].Tag
        (h) Regs[I.Dest].Ready = False                        // value ready only after execution

    else exit loop // stop dispatching if scheduling queue is full
```

Tomasulo algorithm, part 2

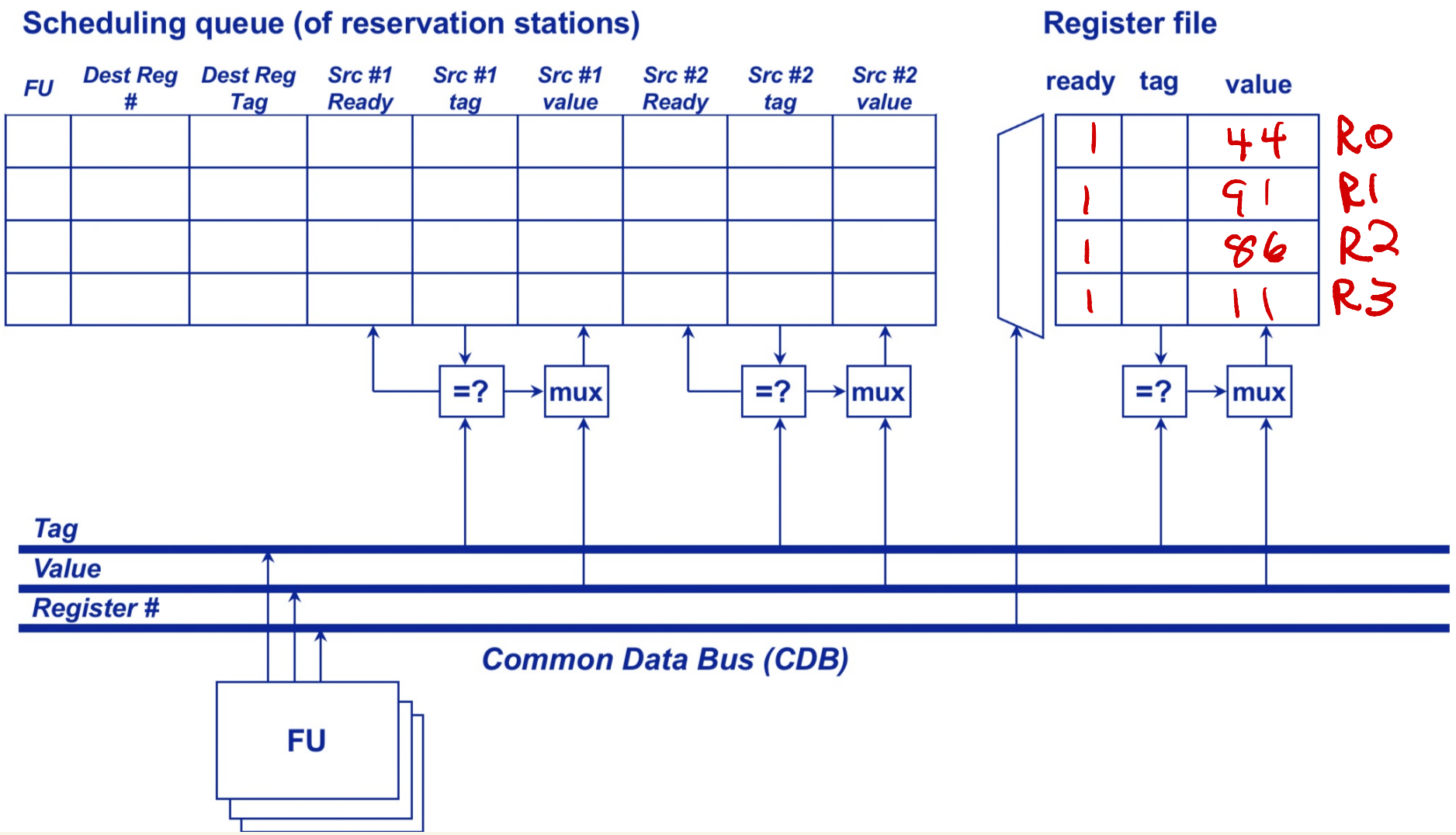
```
[ Scheduling unit ]
for each RS = entry in SchedQueue do:
        (a) for i = 0..1 do:
                if (CDB.Tag = RS.Src[i].Tag) then          // broadcast on CDB matches stored tag
                        RS.Src[i].Ready = True
                        RS.Src[i].Value = CDB.Value
        (b) if (RS.Src[0].Ready = True AND RS.Src[1].Ready = True ) // WAKEUP: If all registers are
ready...
                and Scoreboard[RS.FU].Busy = False) then
                        Scoreboard[RS.FU].Busy = True // ...reserve the FU and issue
                        Issue the instruction at RS on FU
```

step (b) is called "wake-up"
when more than one instruction is ready, picking between them is called "select" (here we're using FIFO as a select algorithm)

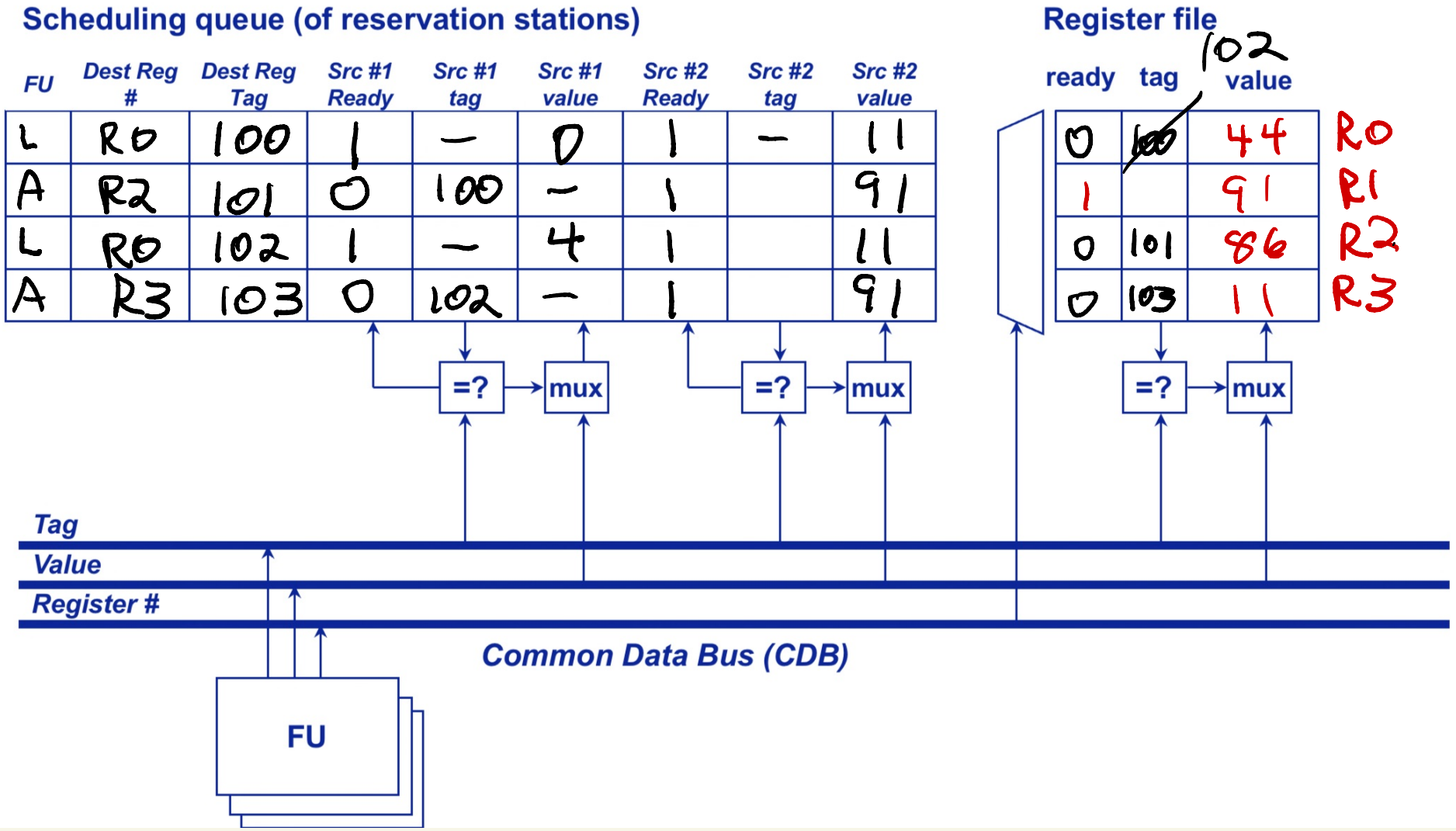
Tomasulo Algorithm, part 3

```
[ Execution unit, for function unit FU]
        if (FU pipeline advances and now first stage is free) then Scoreboard[FU].Busy = False
[ Execution unit, at completion of instruction I ]
        if (CDB.Busy = False) then                          // Broadcast the instruction results
                (a) CDB.Busy = True
                (b) CDB.Tag = I.Tag
                (c) CDB.Value = I.Value
                (d) CDB.Reg = I.Dest
                (e) if (I.FU is not pipelined) then
                        Scoreboard[I.FU].Busy = False
                (f) Delete I from Sched Queue

[ Register file ]
        if (CDB.Busy = True and
        CDB.Tag = Regs[CDB.Reg].Tag) then
                (a) Regs[CDB.Reg].Ready = True              // Update the RF contents
                (b) Regs[CDB.Reg].Value = CDB.Value
                (c) CDB.Busy = False
```



A: LW R0, 0(R3)
B: ADD R2, R0, R1
C: LW R0, 4(R3)
D: ADD R3, R0, R1

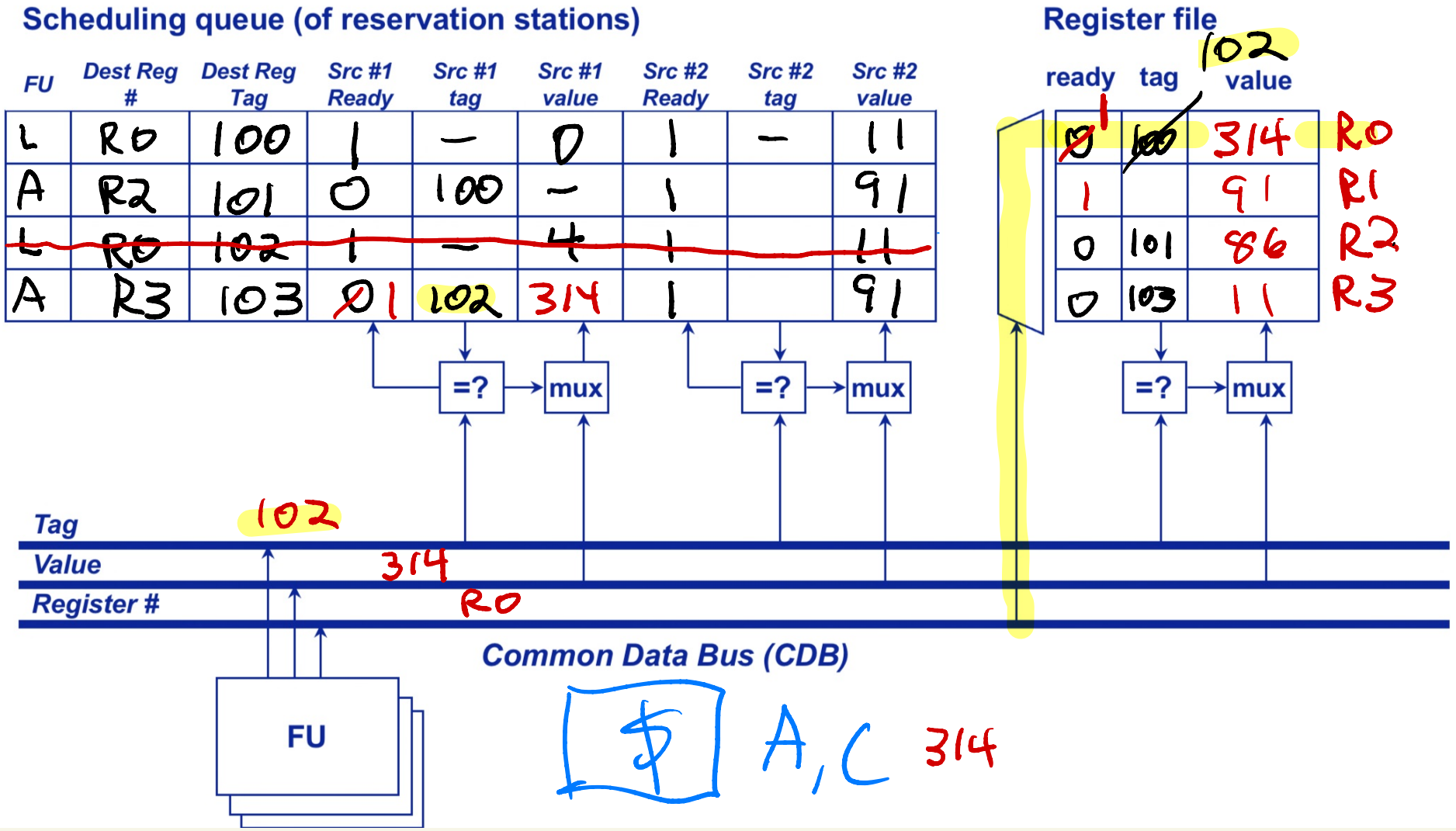


A: LW R0, 0(R3) ✓

B: ADD R2, R0, R1 ✓

C: LW R0, 4(R3) ✓

D: ADD R3, R0, R1 ✓

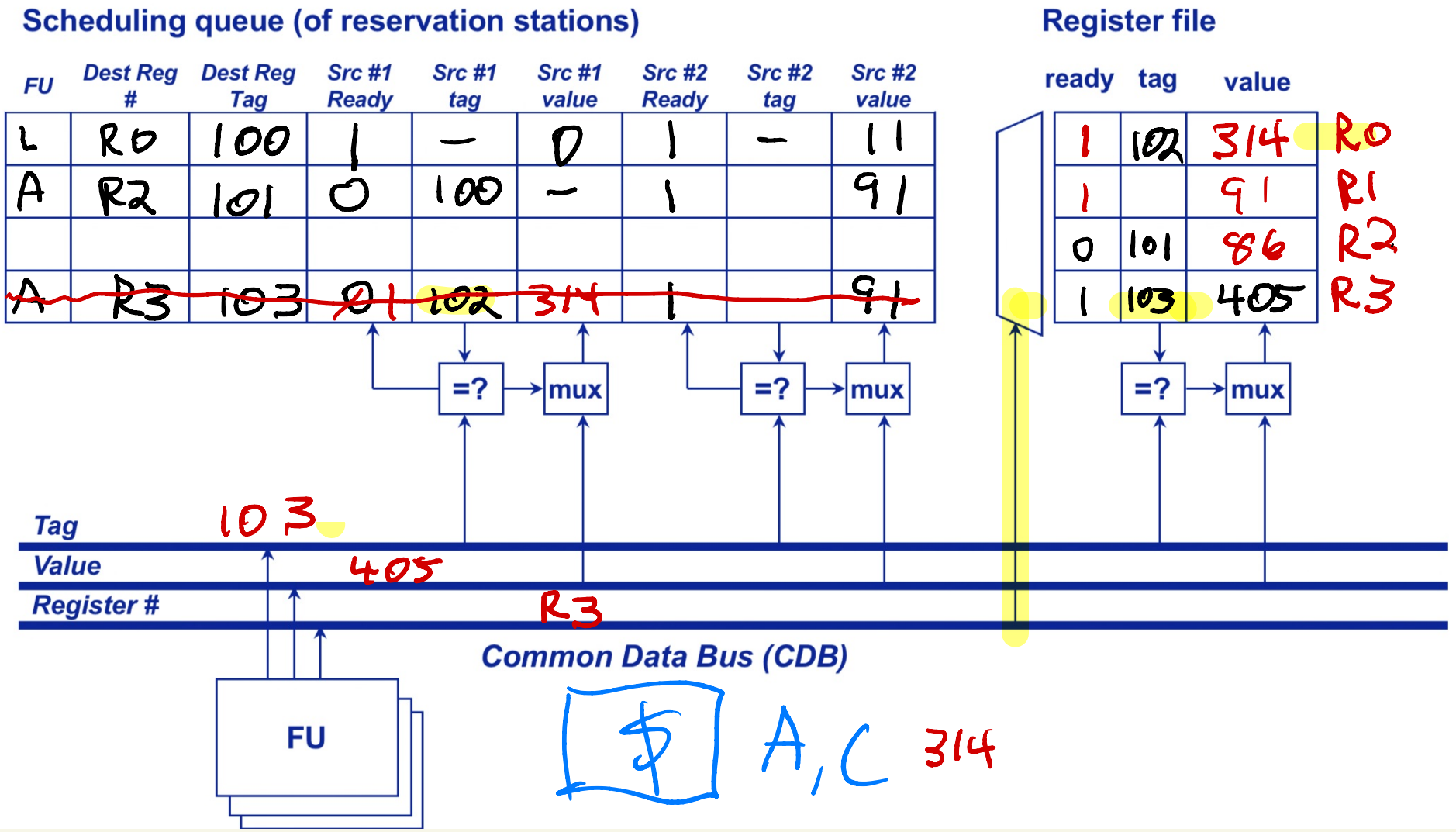


A: LW R0, 0(R3) ✓ cache miss

B: ADD R2, R0, R1 ✓

C: LW R0, 4(R3) ✓ cache hit

D: ADD R3, R0, R1 ✓

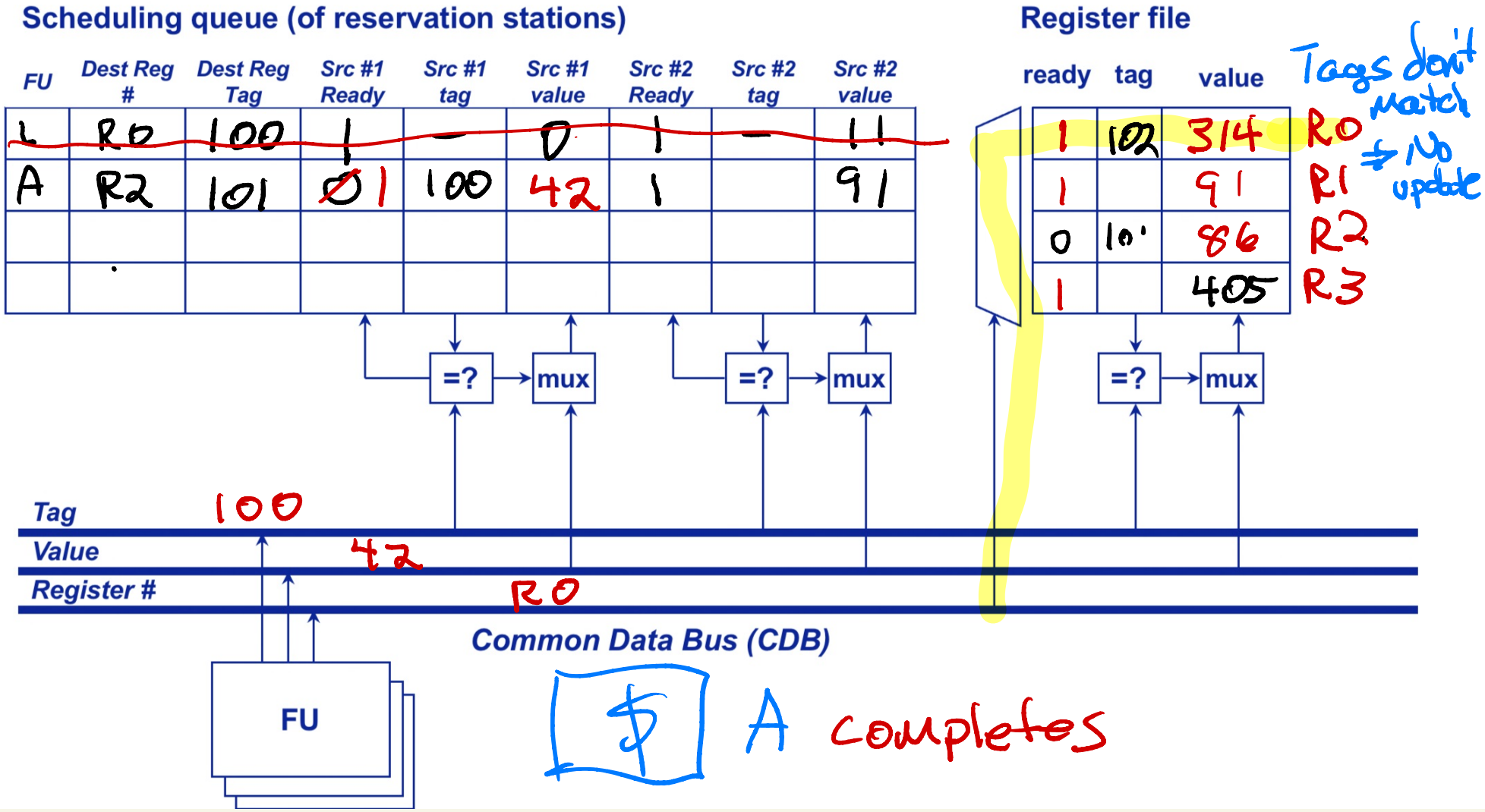


A: LW R0, 0(R3) ✓ cache miss

B: ADD R2, R0, R1 ✓

C: LW R0, 4(R3) ✓ cache hit

D: ADD R3, R0, R1 ✓



A: LW R0, 0(R3) ✓ cache miss → 42

B: ADD R2, R0, R1 ✓

C: LW R0, 4(R3) ✓ cache hit

D: ADD R3, R0, R1 ✓

Tomasulo limitations

HPS$_m$

C.D.B. limits # inst. that can complete each cycle

⇒ more than one CDB

"Result buses"

Best is 1 result bus per FU

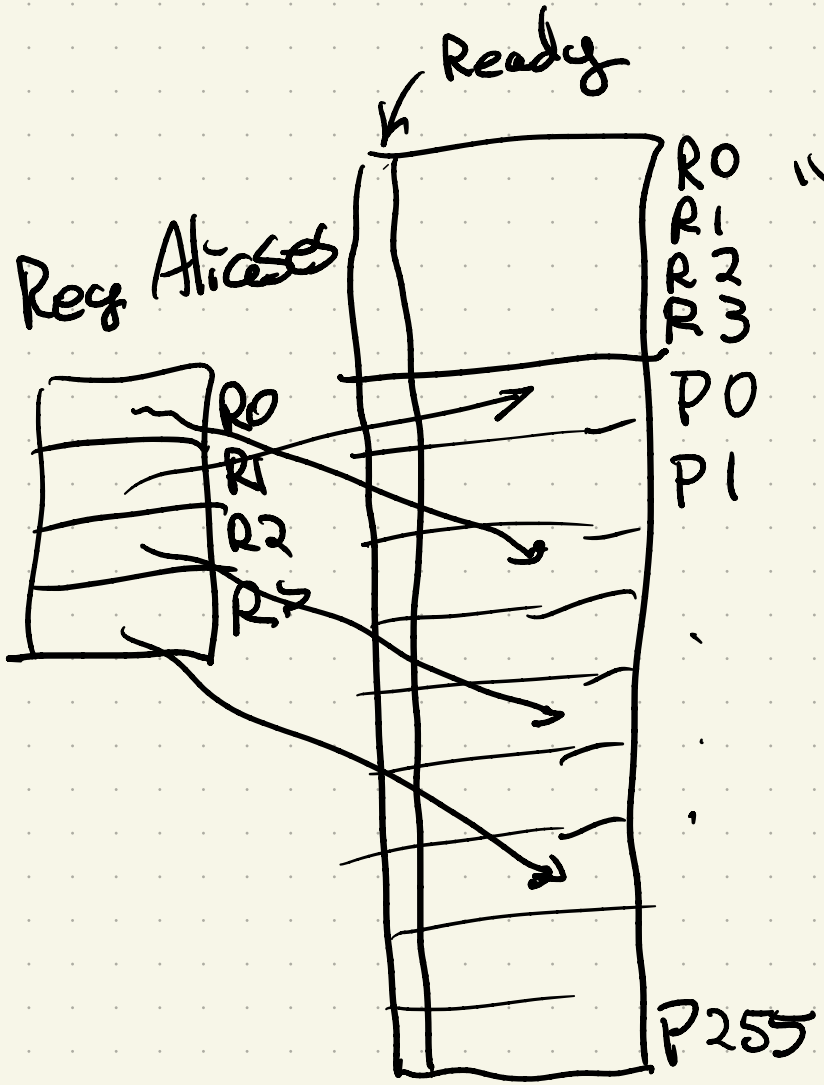
Scheduling queue is complex

Fully associative

Complexity grows with # result buses

Alternative

Remove tags, use large register file



"Architected" (in I.S.A.)

Physical registers

Free list

New Homework

Due 1 week Fri.